Extending AnomalyGFM: What Helps, What Doesn't, and Why

Zero/few-shot graph anomaly detection with a graph foundation model

Term project team

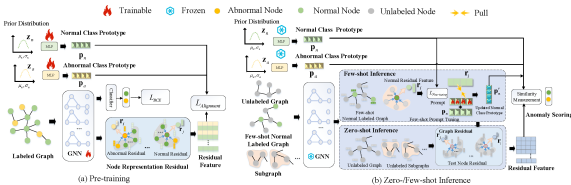
November 11, 2026

Outline

1. The paper
2. Setup
3. Benchmark: scoring and features
4. Four contributions
5. Results
6. Wrap-up

AnomalyGFM (KDD'25): the problem

- **Graph anomaly detection (GAD):** flag nodes unlike the majority (fraud accounts, fake reviews).
- Classical GAD trains and tests on *one* graph $\Rightarrow$ no transfer to unseen graphs.
- Generic graph foundation models fail on GAD: anomalies are rare, irregular, heterogeneous.
- **Goal:** one model, pre-trained once, that detects anomalies in a new graph with zero labels (zero-shot) or a few labelled normal nodes (few-shot).



(a) Pre-training (b) Zero-/Few-shot Inference

How AnomalyGFM scores a node

1. Project every graph to $d' = 8$ dims with a fixed **SVD**; encode with a 2-layer **GCN**.
2. **Residual:** $r_v = h_v - \sum_{u \in \mathcal{N}(v)} w_{vu} h_u$ (small for normal nodes, large for anomalies, in *any* graph).
3. Two learnable **prototypes** p_n, p_a (noise $\rightarrow$ linear+ReLU), trained with labelled anomalies on one graph (Facebook, 30% labelled).
4. **Zero-shot score:** $s_v = \exp(\cos(r_v, p_a))$ — no tuning on the target.
5. **Few-shot:** tune a small prompt with k labelled normal nodes.

Our question

Which parts of this pipeline are weak, and can we improve accuracy, label-efficiency or parameter-efficiency?

Experimental setup (real data only)

- **Train:** Facebook (1,081 nodes, 2.3% anomalies)
- **Test (zero-shot):** Amazon (11,944), Amazon-UPU (10,224), Reddit (10,984)
- **5 seeds**, mean$\pm$std; AUROC and AUPRC
- Same node set for every row (target minus the 100 tuned normal nodes)
- **Also:** in-domain AUROC on held-out Facebook nodes (is the module learning at all?)

Infrastructure

RTX 4500 Ada (24 GB); code local, runs on server via `sync.sh`; sparse GPU inference; harness verified: sparse = dense results (Amazon 0.9026, Reddit 0.5952).

Raw features

Shared files hold only 8-dim SVD; we fetched raw features (labels verified identical) for the hypernetwork.

Weaknesses we looked for

P1 Inference prompts are random noise

P2 Score uses direction only (ignores residual size)

P3 One abnormal prototype for heterogeneous anomalies

P4 Residual = plain neighbour mean (camouflage)

P5 No test-time adaptation (Reddit ≈ 0.57)

P6 Hand-tuned fusion weight in released code

Exp I – is a better *scoring function* available? (10 scorers)

Exp H – is a better *feature alignment* available? (dim $\times$ normalisation, 16 configs)

9 target graphs $\times$ 5 seeds $\times$ 80 trained models = 7,200 result rows.

Exp I: no scorer beats the paper's abnormal score

Scorer	Mean AUROC (9 graphs)	vs paper
abnormal-prototype cosine (paper)	**0.603**	–
rank-fuse(abnormal, residual norm)	0.592	-0.011 (n.s.)
Mahalanobis of residual	0.558	-0.045 (p=0.16)
residual magnitude only	0.536	-0.067
Euclidean to prototypes	0.498	-0.105
paper's *normal* score / code's fusion	0.398	-0.205 (inverted)

- The *normal*-prototype score is worse than chance on 7 of 9 graphs; sign-flipped it ties the abnormal score $\Rightarrow$ the two prototypes are nearly collinear (inferred from AUROC; angle not measured).

Exp H: feature alignment — and a numerical flaw

dim	row-L2	none	row-sum (paper)	z-score
2	.576	.571	.550	.509
4	.616	.596	.571	.564
6	.615	.598	.548	.549
8	**.622**	.609	.603	.573

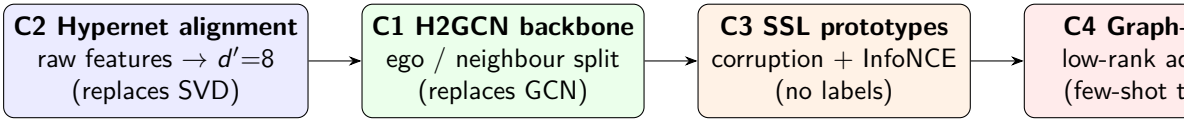
mean AUROC over 9 targets, 5 seeds

- ≥ 4 dims suffice; z-score is worst.
- Row-L2 beats the paper's normalisation on average, but **not significantly** ($p = 0.91$ over graphs; 4/9 graphs better) — gain is all Elliptic.
- Paper divides by the *signed* row sum of SVD features: 52.6% (Elliptic) to ∼99.5% (Amazon) of rows have negative sums ⇒ sign flips and values up to $\sim 10^4$.

Take-away

Cheap re-scoring / re-normalising does not help. The lever is *what the model learns*.

Four extensions, one per team member



pre-train on Facebook $\Rightarrow$ zero-shot / few-shot on unseen graphs

C1: H2GCN – heterophily-aware backbone

Why: GCN's self-loop averages a node into its neighbours, shrinking the residual the detector needs.

How (H2GCN principles): ego/neighbour separation, 1- and 2-hop, combine:

$$\text{out} = \sigma(XW_e + A_n XW_1 + A_n^2 XW_2 + b),\ A_n = \hat{A} - I$$

Drop-in for the paper's GCN; params 484k$\rightarrow$ 811k.

AUROC	Amazon	UPU	Reddit
Baseline	**.834**	**.792**	.564
+C1	.668	.701	.589

- Learns in-domain (FB-val .789 vs .828) but transfers worse.
- Reddit +0.025 is within noise.
- Seed std 3$\times$ the baseline's.

C2: hypernetwork feature alignment

Why: SVD is linear and fitted per graph; its components mean different things in different graphs.

How: a hypernetwork generates a 2-layer non-linear projection from *feature descriptors* (mean, skew, graph smoothness, ...) and *graph metadata* (size, density, ...); works for any width D.

$$W_1[j] = g_1([z_j; m]),\ W_2 = g_2(m)$$

23k extra parameters; trained end-to-end.

AUROC	Amazon	UPU	Reddit
Baseline	**.834**	**.792**	.564
+C2	.418	.398	.584

- Below chance on Amazon (ranking inverted).
- In-domain it learns (.750).
- Likely cause: one training graph = one metadata point; nothing teaches how the projection should vary (*untested*).

C3: contrastive prototype learning with no labels

Why: the paper needs labelled anomalies; they are scarce.

How: corrupt the training graph each epoch (rewire 5% of neighbourhoods, swap features of 5% of node pairs) $\Rightarrow$ corrupted nodes = pseudo-abnormal.

- **Prototype InfoNCE:** residual must pick its pseudo-class prototype (τ=0.2, class-balanced)
- **View InfoNCE:** clean-view node retrieves itself in the corrupted view

AUROC	Amazon	UPU	Reddit
Supervised baseline	**.834**	**.792**	.564
C3, no labels	.620	.588	.585

- **0.62 AUROC with zero labels** (chance = 0.5); in-domain .701.
- Gap to supervision: random corruption $\neq$ camouflaged fraud.
- Can use *any* unlabelled graph $\rightarrow$ multi-source.

C4: Graph-LoRA for few-shot adaptation

How: freeze the GNN; learn $W' = W + \frac{\alpha}{r}BA$ (r=4, B=0 at init); toggle on/off.

Method	Trainable params
Prompt (paper idea)	80
Graph-LoRA	**4,832**
Full GNN fine-tune	164,002

LoRA = 2.9% of full fine-tuning.

AUROC	Amazon	UPU	Reddit
zero-shot baseline	.834	.792	.564
LoRA (paper budget)	.834	.793	.563
Full FT (paper budget)	.839	.795	.563
LoRA (30 ep, 1e-3)*	.651	.693	.608
Full FT (30 ep, 1e-3)*	.637	.586	.596
Prompt (30 ep, 1e-3)*	.851	.797	.568

*sensitivity check, examined after seeing results.

Honest reading

Efficiency claim holds; accuracy does not improve. Released few-shot code leaves its prompt commented out of `forward` — it does no real tuning.

Main results: single training graph (Facebook)

Method	Params	Amazon	Amazon-UPU	Reddit
Baseline AnomalyGFM	484,402	**.834**$_{\pm.022}$	**.792**$_{\pm.017}$	.564$_{\pm.026}$
+ C1 H2GCN	810,802	.668$_{\pm.060}$	.701$_{\pm.051}$	.589$_{\pm.044}$
+ C2 Hypernet	507,514	.418$_{\pm.092}$	.398$_{\pm.055}$	.584$_{\pm.043}$
+ C3 SSL (*no labels*)	484,002	.620$_{\pm.082}$	.588$_{\pm.056}$	.585$_{\pm.010}$
C1+C2+C3 combined	833,914	.529$_{\pm.105}$	.543$_{\pm.030}$	.550$_{\pm.016}$

- AUROC, mean±std over 5 seeds. **No extension beats the baseline on Amazon.**
- Reddit differences (≤ 0.025) are within one std.
- Peak GPU memory: extensions add only 4–5% (Amazon: 508 $\rightarrow$ 511–533 MB).

Do the modules learn? In-domain vs. transfer

Variant	Held-out Facebook AUROC	Amazon AUROC
Baseline	0.828	0.834
C1 H2GCN	0.789	0.668
C2 Hypernet	0.750	0.418
C3 SSL (no labels)	0.701	0.620

- Every module **learns** (all $\gg 0.5$ in-domain) $\Rightarrow$ not a coding bug.
- The failure is **transfer** from a single 1,081-node training graph (about 25 anomalies).
- Next: does multi-source pre-training fix it?

Multi-source pre-training (target domain held out; 3 seeds)

AUROC	Amazon	Amazon-UPU	Reddit
1-source (Facebook) baseline	**.832**	**.785**	.572
4-source baseline	.611	.612	.591
4-source + C1 H2GCN	.548	.548	.590
4-source + C2 Hypernet	.620	.617	.467
4-source + C3 SSL (*no labels*)	.573	.576	**.610**
4-source C1+C2+C3	.530	.508	.476

- Pool: Facebook, Reddit, Amazon, YelpChi, T-Finance minus the target's domain; large graphs as BFS subgraphs ($\leq$4,000 nodes).
- **Naive pooling hurt the supervised baseline** on Amazon (-0.22) — negative transfer; Reddit $+0.02$ (noise).

Reading the multi-source result

What the data support

- C2's inversion disappears with 4 metadata points ($0.36 \rightarrow 0.62$): supports "one graph = one metadata point".
- C3 is the best method on Reddit (0.610, label-free) — suggestive, 3 seeds.
- Adaptation again: no-op at paper budget; LoRA/full hurt on Amazon (≈ 0.45), LoRA most stable on Reddit ($\pm .007$).

What we cannot claim

- C2 still only *matches* the degraded baseline and gets worse on Reddit.
- Why pooling hurts is untested: differing anomaly semantics? BFS-subgraph statistics $\neq$ full graph? our own SVD for Yelp/T-Fin? equal source weights?
- It is "naive pooling *with our sampling*" that fails — not multi-source pre-training in general.

What we learned

1. **The paper reproduces** (Amazon $\approx$ 0.83–0.86) and our harness is exact (sparse = dense).
2. **Cheap fixes don't work:** no re-scoring or re-normalising beat the paper (Exp H/I).
3. **Prototype problem:** normal/abnormal prototypes are nearly collinear; the paper's normal score is inverted.
4. **C3 is the useful idea:** anomaly structure can be learned with zero labels.
5. **C4 gives efficiency, not accuracy:** 4,832 params vs 164,002, same accuracy at the paper's budget.
6. **C1/C2 do not transfer** from one small training graph — negative results; **multi-source pooling** removes C2's inversion but hurts the baseline.
7. **Released few-shot code is a no-op** (prompt commented out).

Limitations and next steps

Limitations

- One training graph (paper protocol), 5 seeds, 3 targets
- No re-tuning of the larger models; we avoided tuning on target graphs (leakage)
- Explanations of *why* are hypotheses unless marked tested

Next steps

- Camouflage-aware corruptions for C3
- Use the abnormal prototype / pseudo-anomalies in few-shot objective (C4)
- Prototype-separation loss (fix collinearity)
- LoRA rank sweep; hypernet with a learned feature-set encoder

Code & reports

github.com/SumitKumar-17/AnomalyGFM (branch `team-contributions`): `WORK.md`, `reports/C1-C4`, `evaluate_team_contributions.py`